

EX 1 -Clustering based data analytics using R/Python

AIM :

To perform clustering-based data analytics using K-Means and Self-Organizing Map (SOM) algorithms in R/Python, in order to group similar data points into meaningful clusters.

ALGORITHM :

1. K-Means Clustering

1. Choose the number of clusters K
2. Initialize K centroids randomly
3. Assign each data point to the nearest centroid
4. Recalculate centroids by taking the mean of assigned points
5. Repeat steps 3–4 until centroids do not change
6. Output the final clusters

2. Self-Organizing Map (SOM)

1. Initialize a grid of neurons with random weights
2. Select an input data point
3. Find the Best Matching Unit (BMU) (closest neuron)
4. Update weights of BMU and its neighbors
5. Reduce learning rate and neighborhood size over time
6. Repeat for all data points
7. Output clustered map representation

CODE :

```
!pip install Minisom

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

from sklearn.preprocessing import MinMaxScaler

from minisom import MiniSom


iris = load_iris()

X = iris.data

y = iris.target


scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)


kmeans = KMeans(n_clusters=3, random_state=42)

kmeans_labels = kmeans.fit_predict(X_scaled)


som = MiniSom(x=3, y=1, input_len=4, sigma=1.0, learning_rate=0.5, random_seed=42)

som.random_weights_init(X_scaled)

som.train_random(X_scaled, 500)


som_labels = []

for x in X_scaled:

    winner = som.winner(x)

    som_labels.append(winner[0])
```

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
```

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=kmeans_labels, cmap='viridis')
```

```
plt.title("K-Means (Scaled Data)")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.subplot(1,2,2)
```

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=som_labels, cmap='viridis')
```

```
plt.title("SOM (Same Scale)")
```

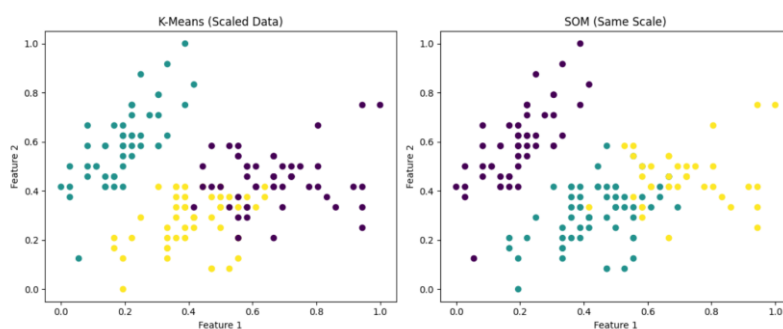
```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT :



RESULT :

Thus the above program is executed and the result is obtained.